PyPyrus — A Data Provenance Layer for Transparent and Reproducible Machine Learning Systems

Project structure

```
pypyrus
├─ docs
├─ examples
├─ experiments
├─ pypyrus
│  ├─ core               # Core abstractions (run, dataset identity, etc.)
│  ├─ instrumentation    # Library-specific instrumentation (PyTorch, TF,
JAX)
│  ├─ provenance         # Event schemas + provenance semantics
│  ├─ reporting          # Query + report generation logic
│  ├─ storage            # Store implementations (e.g. SQLite)
│  └─ utils              # Helper functions
├─ scripts
├─ tests
├─ README.md
└─ pyproject.toml
```

Core Abstractions

1. Run

A **Run** is the container that groups all provenance events for one training execution.

Fields

- `run_id` (UUID)
- `start_time`, `end_time`
- `code_ref` (git commit hash + dirty flag)
- `config_ref` (hash of config dict/file)
- `environment_ref` (hash of environment snapshot)
- `tags` (freeform labels)

Responsibilities

- Own a `Store`
- Attach instrumentation
- Emit `RunStartEvent` and `RunEndEvent`
- Ensure flush/close on exit (context manager)

The Run provides the **audit boundary**.

---

2. Dataset Identity

We must be able to state exactly **what dataset** was used, even if not explicitly declared in code.

DatasetDescriptor

- `dataset_id` (stable internal ID)
- `name` (human readable)
- `uri` (path, S3 URL, registry reference, etc.)
- `version_hint` (optional; HF revision, DVC commit, etc.)

DatasetFingerprint

A stable identifier of dataset state.

Strategies:

- Manifest hash (file paths + sizes + mtimes)
- Sampled content hash (chunk-based)
- Full content hash (optional strong mode)
- Registry-provided version ID

Goal:

> Detect dataset changes even if the path remains the same.

Provenance Event Schema

Everything logged during a run is represented as an event. Events are **small, append-only, and cheap**.

Event Types

1️⃣ RunStartEvent

Emitted when a run begins.

Fields

- `run_id`
- `timestamp`
- `code_ref`
- `config_ref`
- `environment_hash`
- `seed_summary`

Purpose:

- Anchor all other events
- Enable reproducibility bundle

2️⃣ RunEndEvent

Emitted when a run ends.

Fields

- `run_id`
- `timestamp`
- `status` (success/failure)
- `event_count` (optional)

Purpose:

- Close the audit boundary
- Support lifecycle traceability

3️⃣ DatasetRegisteredEvent

Emitted when a dataset is wrapped or first accessed.

Fields

- `run_id`
- `dataset_id`
- `name`
- `uri`
- `version_hint`
- `fingerprint`
- `fingerprint_method`

Purpose:

- Bind human dataset identity to a stable fingerprint
- Enable dataset version traceability (AI Act Art. 10)

4️⃣ TransformDeclaredEvent

Emitted when transform pipelines are defined or attached.

Fields

- `run_id`
- `dataset_id`

- `transform_chain_id`
- `transform_list` (ordered names)
- `params_hash`
- `deterministic_flag`
- `seed_policy` (global/per-worker/per-sample)

Purpose:

- Record preprocessing and augmentation logic
- Support transparency and reproducibility
- Avoid logging per-sample transform noise

Purpose:

- Track dataset usage frequency
- Detect imbalance / oversampling
- Provide dataset usage statistics

6️⃣ BatchDeliveredEvent (Primary Reproducibility Event)

Emitted when a batch is received by the training loop.

This defines the **global training order**.

Fields

- `run_id`
- `dataset_id`
- `global_step` (monotonic)
- `batch_size`
- `batch_fingerprint` (ordered hash of sample IDs)
- `sample_ids` (optional; stored in debug/full mode)
- `rng_state_hash` (optional)

Purpose:

- Compare batch streams across runs
- Detect divergence
- Reconstruct training order under shuffling
- Enable data-stream reproducibility experiments

This event answers:

> What did batch t contain in run A vs run B?

Optional Strict version — BatchConsumedEvent (Step Boundary)

This is emitted inside a wrapped training step function, immediately before or after the model forward pass.

This mode requires minimal additional integration (e.g., wrapping a `train_step` function).

In strict version:

- `global_step` aligns with training steps
- Skipped batches are not logged
- Logging reflects actual computation entry

7️⃣ EnvironmentSnapshotEvent (optional)

Emitted once at run start.

Fields

- `python_version`
- `library_versions_hash`
- `hardware_summary`
- `cuda_version` (if applicable)

Purpose:

- Strengthen reproducibility evidence
- Support documentation generation

Instrumentation Layer

Instrumentation defines **where events are emitted** in the training flow.

Flow Mapping

Training Flow Step	Event Emitted
`start_run()`	RunStartEvent
Dataset registered / wrapped	DatasetRegisteredEvent
Transform introspected at registration	TransformDeclaredEvent
DataLoader yields batch to training loop	BatchDeliveredEvent
`end_run()`	RunEndEvent

PyPyrus captures provenance at the **DataLoader → training loop handoff**, avoiding intrusive modification of model or optimizer logic.

Optional Strict Flow (Step Boundary Mode)

Training Flow Step	Event Emitted
`start_run()`	RunStartEvent

Training Flow Step	Event Emitted
Dataset registered	DatasetRegisteredEvent
Transform introspected	TransformDeclaredEvent
Batch yielded	(IDs attached only)
Training step executed	BatchConsumedEvent
`end_run()`	RunEndEvent

In this mode:

- DataLoader wrapper propagates sample IDs (BatchDeliveredEvent)
- Step wrapper emits the event (BatchConsumedEvent)

Instrumentor Interface

```
attach_dataloader(loader) -> loader_proxy
```

Responsibilities:

- Wrap DataLoader iterator
- Ensure sample IDs are propagated per batch
- Compute ordered batch fingerprint
- Emit BatchDeliveredEvent
- Buffer events for efficient storage writes

Optional batch-consumed instrumentation:

```
wrap_step(train_step) -> instrumented_step
```

Responsibilities:

- Emit BatchConsumedEvent at computation boundary
- Maintain accurate global_step alignment

Store (Local Provenance Store)

Interface:

- `append_event(event)`
- `flush()`
- `query_events(...)`

Baseline implementation:

- SQLite

Design properties:

- Append-only
- Indexed by run_id, dataset_id, event_type
- Schema versioned

Query & Reporting Layer

Transforms raw provenance into meaningful outputs.

Query Examples

- Which datasets were used in run X?
- What was the delivered batch sequence?
- At what step did batch streams diverge?
- What did batch 127 contain?
- What is the match rate between run A and B?

Report Builder Outputs

Dataset Traceability

- Dataset identity + fingerprint
- Split information (if available)

Data Stream Summary

- Total batches delivered
- Match rate across runs
- First divergence step
- Batch size consistency

Transform Chain

- Ordered preprocessing pipeline
- Parameter hashes

Reproducibility Bundle

- Dataset fingerprint
- Code reference
- Config hash
- Seeds

- Batch stream fingerprint summary

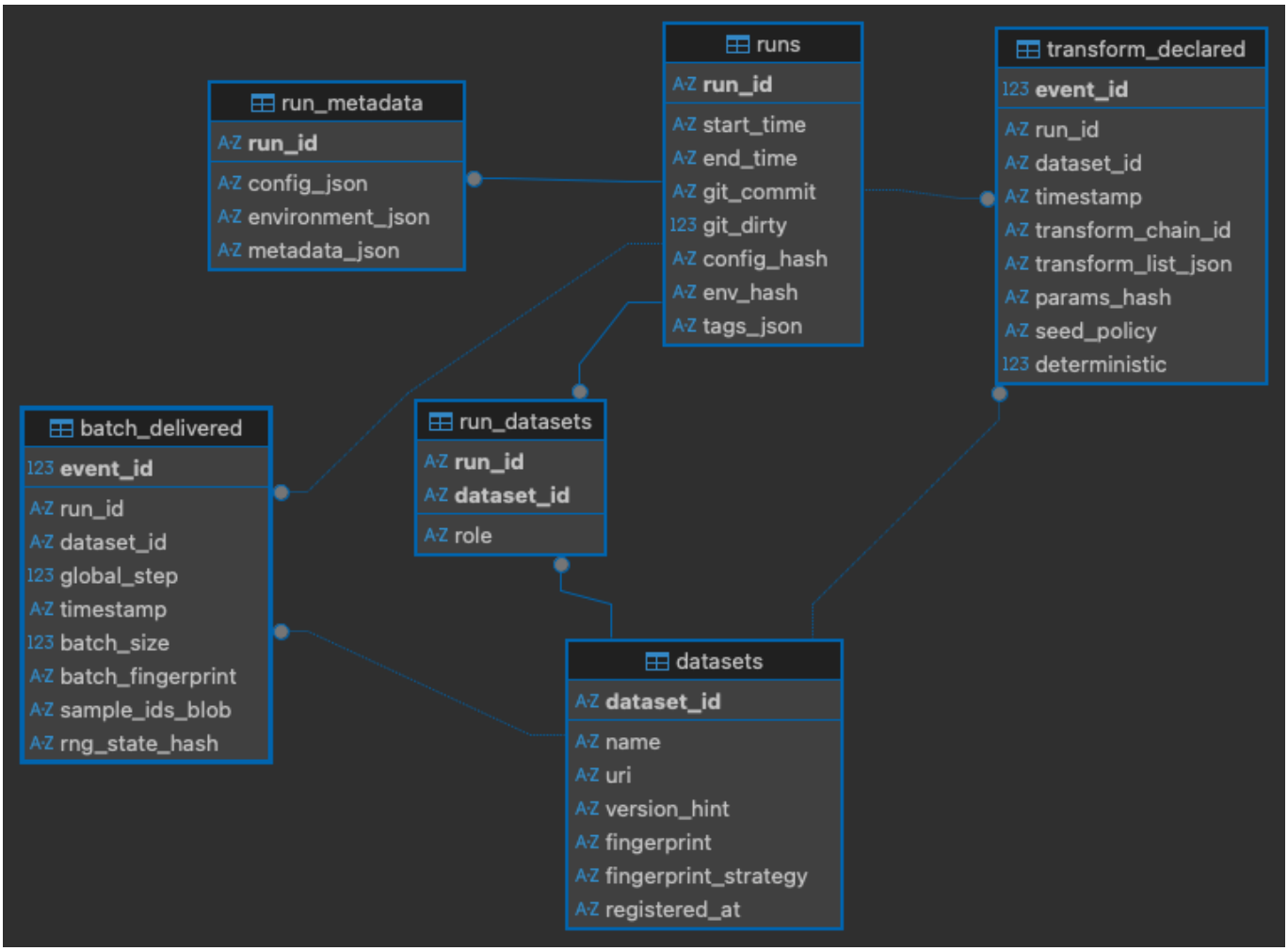
Design Principles

- Events represent **evidence**, not every micro-operation
- Logging is proportionate and configurable
- Reproducibility is evaluated at the **data stream level**
- Dataset identity and batch order are first-class citizens
- The system supports compliance evidence without enforcing policy

---

Entity Relationship Diagram

The following ER diagram illustrates the core tables and relationships in the PyPyrus schema. The db schema is subject to change as we iterate, and a few fields are still TBD (e.g. seed policy, deterministic flag, etc.) but this should give a good overview of the main entities and their connections.



What Do We Mean by “Reproducibility”?

Reproducibility is not one thing. There are **levels**.

Level 0 — Output Reproducibility

> Same model weights, same metrics.

This depends on:

- GPUs / cuda kernels
- floating point nondeterminism
- hardware

PyPyrus does **not** guarantee this.

Level 1 — Configuration Reproducibility

> Same code, same config, same seeds.

The `runs` table supports this:

- git commit
- config_hash
- env_hash
- seed summary

But this is kinda weak — people already log configs.

Level 2 — Dataset Identity Reproducibility

> The same dataset version was used.

Pypyrus `datasets` table supports:

- dataset fingerprint
- version_hint (e.g. HF revision or DVC commit)

WandB and others can log dataset versions, but they often rely on user input.

Level 3 — Data Stream Reproducibility (Core Claim)

> The model saw the same sequence of batches in the same order.

This is where the schema shines.

From `batch_delivered` (or `batch_consumed` in strict version):

- `global_step`
- `batch_fingerprint`

You can:

- Compare runs A and B step-by-step
- Compute match rate
- Find first divergence step
- Detect shuffling differences
- Detect dropped batches (missing steps)

This is **reproducibility of the training data stream**.

This is a strong, measurable claim that goes beyond config logging. It's about the actual data seen by the model.

Level 4 — Exact Batch Reconstruction

(Optional, We could maybe implement this and somehow compress the sample_ids if it doesn't blow up storage)

If we store `sample_ids`:

We can:

- Show exactly which samples were in batch 127
- Replay the exact data stream (a task for the future perchance)
- Debug divergence precisely

That's full data-stream replay capability!

So To What Extent Can We Show Reproducibility?

With your current schema:

We can prove:

- Same dataset version
- Same transform pipeline declaration
- Same batch sequence
- Same batch grouping
- Same divergence point

We cannot prove or wont attempt at least to prove:

- Same floating point execution
- Same internal augmentation randomness
- Same gradient values

How we can demonstrate this:

In experiments:

1. Run training twice under fixed seed.
2. Compare:
    - dataset fingerprint
    - transform params_hash
    - batch_fingerprint per step
3. Show 100% match.

Then:

4. Introduce "extreme" shuffling.
5. Show:
    - Divergence at step 37.
    - Different batch_fingerprint.
    - Possibly different sample usage distribution.

This becomes measurable evidence.

We are not claiming:

> PyPyrus guarantees identical model weights.

But we are claiming:

> PyPyrus enables reproducibility at the data-stream level by making the training data sequence observable, comparable, and "reconstructable" at some point (seems tedious to implement exact batch replay, but thats another project perchance).

That is precise, defensible, and aligned with AI Act traceability requirements.

PyPyrus defines dataset-level reproducibility at the data stream boundary rather than at raw dataset access or internal model computation. This avoids logging execution-layer artifacts (e.g., worker scheduling or prefetching) and instead records the exact sequence of samples delivered to training.

Instrumentation Architecture

PyPyrus instruments the training pipeline at two boundaries:

1. **Dataset boundary** — inject stable sample identifiers
2. **DataLoader boundary** — observe and log the delivered batch stream

Instrumentation is part of the `instrumentation/` module:

```
pypyrus/
  instrumentation/
    dataset.py      # Inject sample IDs
    dataloader.py   # Observe batch stream + emit events
    collate.py      # Preserve IDs through collation (helper)
```

Dataset Wrapper (`dataset.py`)

Boundary: `dataset.__getitem__`

The dataset wrapper injects a stable `sample_id` into every returned sample.

Responsibilities:

- Attach deterministic `sample_id` (default: dataset index)
- Preserve original dataset behavior
- Perform no logging

This ensures identity propagates through batching and multiprocessing.

DataLoader Wrapper (`dataloader.py`)

Boundary: DataLoader iterator (`for batch in loader:`)

The DataLoader wrapper:

- Extracts ordered `sample_ids` per batch
- Computes a batch fingerprint
- Emits `BatchDeliveredEvent`
- Buffers events for efficient storage

A `BatchDeliveredEvent` means:

> This batch was delivered from the DataLoader to the training loop.

This defines the observable **data stream boundary** used for reproducibility analysis.

Collate Handling (`collate.py`)

Because batches may have arbitrary structure (tuples, dicts, nested objects, custom `collate_fn`), PyPyrus optionally wraps the `collate_fn` to ensure `sample_id`s are preserved during collation.

The original batch structure is preserved for user code.

Public API

Instrumentation requires a single call:

```
loader = pypyrus.attach(loader)
```

Internally this:

1. Wraps the dataset
2. Wraps the DataLoader
3. Enables batch-level provenance logging

No changes to model or optimizer code are required in default mode.
